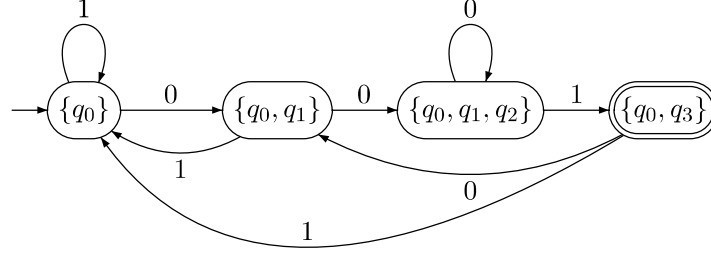


# Computability 2024 - Exam Moed A Solution

## Solution 1:



## Solution 2:

The claim is correct. Let  $w = \sigma_1 \cdot \sigma_2 \cdots \sigma_k$  be a word of length  $k \geq |Q|$  for which there exists an accepting run  $r = q_0, q_1, \dots, q_k$ , where  $q_k \in F$ . For  $0 \leq i < j \leq k$  let  $w[i, j] = \sigma_i \cdot \sigma_{i+1} \cdots \sigma_j$ , and for  $j < i$  let  $w[i, j] = \varepsilon$ . Since,  $k \geq |Q|$  and the run  $r$  consists of  $k + 1$  states, there exist  $0 \leq i < j \leq k$  such that  $q_i = q_j$ . For every  $t \geq 1$  consider the word  $w_t = w[1, i] \cdot w[i + 1, j]^t \cdot w[j + 1, k]$ . Note that  $|w_t| = k + (t - 1)|j - i|$ . We claim that  $w_t \in L(A)$  for all  $t \geq 1$  and since the length of  $w_t$  strictly increases with  $t$  we conclude that  $L(A)$  is infinite. Observe that by definition of  $r$  we have  $q_i \in \delta^*(q_0, w[1, i - 1])$ ,  $q_i = q_j \in \delta^*(q_i, w[i, j])$ , and  $q_k \in \delta^*(q_j, w[j + 1, k])$ . It follows by induction that  $q_k \in \delta^*(q_0, w_t) \cap F$ , and so  $w_t \in L(A)$  for all  $t \geq 1$ .

## Solution 3:

The claim is incorrect. Consider for example the NFA  $A = \langle \Sigma, Q, Q_0, \delta, F \rangle$ , where  $\Sigma = \{1\}$ ,  $Q = Q_0 = \{q_1, q_2\}$ ,  $F = \{q_1\}$  and  $\delta(q, 1) = \{q\}$  for all  $q \in Q$ . Clearly every word  $w \in \Sigma^*$  has exactly two runs. Each obtained with one of the self loops, one from  $q_1$  to  $q_1$ , and the other from  $q_2$  to  $q_2$ . Accordingly, the language of  $A$  is  $\Sigma^*$  no matter which of the sets  $F = \{q_1\}$ , or  $Q \setminus F = \{q_2\}$  is used as the set of accepting states. Formally, if we let  $B = \langle \Sigma, Q, Q_0, \delta, Q \setminus F \rangle$  then  $L(B) = L(A)$ , and hence  $B$  is not an NFA for the complement language of  $A$ .

## Solution 4:

The language belongs to  $\text{coRE} \setminus \text{R}$ .

**First direction:**  $L \notin \text{RE}$

We prove that  $\text{NE}_{TM} \leq_m \bar{L}$ , and hence by the reduction theorem and the fact that  $\text{NE}_{TM} \notin \text{coRE}$ , we conclude that  $\bar{L} \notin \text{coRE}$ , i.e.  $L \notin \text{RE}$ .

**Construction:** Given an input  $\langle M \rangle$  for  $\text{NE}_{TM}$  we output the pair  $\langle M, M_{all} \rangle$ , where  $M_{all}$  is a TM with the same input alphabet as  $M$  and with an initial state that is also the accepting state.

**Correctness:** Since the initial state of  $M_{all}$  is also the accepting state, and  $M_{all}$  and  $M$  share the same input alphabet  $\Sigma$ , it follows that  $L(M_{all}) = \Sigma^*$  and hence  $\langle M \rangle \in \text{NE}_{TM}$  iff  $L(M) \neq \emptyset$ , iff  $L(M) \cap L(M_{all}) \neq \emptyset$ , iff  $\langle M, M_{all} \rangle \in \bar{L}$ .

**Computability:** To compute  $\langle M, M_{all} \rangle$  we only need to compute  $M_{all}$  and write it after the input  $\langle M \rangle$  (with some separating symbol). The encoding of  $M_{all}$  can be computed for example by copying the encoding  $M$  and only replacing the encoding of the accepting state to be that of  $q_0$ , and replacing the encoding of the rejecting state to some other state. Any other construction is of course valid as long that you take care that  $M_{all}$  has the same input alphabet as  $M$ .

**Second direction:**  $L \in \text{coRE}$

We prove that there exists a TM  $M$  with  $L(M) = \bar{L}$  and so  $L \in \text{coRE}$ . The TM  $M$  works on input  $\langle M_1, M_2 \rangle$  as follows. Let  $\Sigma_1$  and  $\Sigma_2$  be the input alphabets of  $M_1$  and  $M_2$  respectively.  $M$  first verify that  $\Sigma_1 = \Sigma_2$ . Then,  $M$  operates in iteration beginning with iteration  $i = 0$ . In iteration  $i$  the machine  $M$  goes over all words  $x \in \Sigma_1^*$  of length at most  $i$ , and for every such  $x$  it runs  $M_1$  and  $M_2$  over  $x$  for at most  $i$  steps. If there was some  $x$  that both machines accepted then  $M$  accepts. Otherwise,  $M$  proceeds to the next iteration.

**Correctness:** First, note also that if  $M_1$  and  $M_2$  accept  $x$ , then there exists  $i \geq |x|$  such that both  $M_1$  and  $M_2$  accept  $x$  in at most  $i$  steps (Take  $i$  to be the maximum between  $|x|$  and the times it take  $M_1$  and  $M_2$  to accept  $x$ ). So,  $L(M_1) \cap L(M_2) \neq \emptyset$  iff there exists  $i \geq 0$  and  $x \in \Sigma_1^*$  such that  $i \geq |x|$  and both  $M_1$  and  $M_2$  accept  $x$  in at most  $i$  steps. The latter is clearly equivalent to  $\langle M_1, M_2 \rangle \in L(M)$ , and so  $L(M) = \langle \langle M_1, M_2 \rangle : L(M_1) \cap L(M_2) \neq \emptyset \rangle = \bar{L}$ .<sup>1</sup>

**Computability:** The machine  $M$  is well defined and all its functionalities are indeed computable. We saw that it is possible to enumerate  $\Sigma^*$  in minlex order, and we saw that there exists a universal TM  $U$  that on input  $\langle T, w, t \rangle$  simulates the run of a TM  $T$  on the input word  $w$  for  $t$  steps. So in every iteration  $i \geq 0$ , and for every  $x \in \Sigma_1^*$  of length at most  $i$ , the TM  $M$  call  $U$  twice, once on  $\langle M_1, x, i \rangle$  and once on  $\langle M_2, x, i \rangle$ , computes the “AND” result of the two simulations, accepts if the AND result is True, and otherwise increments  $i$  and proceeds to the next iteration.

## Solution 5:

The claim is correct. We saw that  $ALL_{NFA} \in \text{PSPACE}$ , in particular  $ALL_{NFA} \in \text{R}$ . We use this fact to reduce  $ALL_{NFA}$  to  $VC$  as follows.

**Construction:** We define the reduction  $f$  on input  $\langle A \rangle$  for  $ALL_{NFA}$  as follows. Let  $G = \langle V, E \rangle$  be a triangle graph. I.e.,  $V = \{1, 2, 3\}$  and  $E = \{\{1, 2\}, \{2, 3\}, \{3, 1\}\}$ . If  $\langle A \rangle \in ALL_{NFA}$  then  $f(\langle A \rangle) = \langle G, 2 \rangle$ , and otherwise  $f(\langle A \rangle) = \langle G, 1 \rangle$ .

**Correctness:** Clearly the minimal vertex cover of  $G$  is of size 2: Every singleton  $\{v\}$  misses the edge between the other two vertices in  $V \setminus \{v\}$ , and the set  $\{1, 2\}$  is a vertex cover. So indeed,  $\langle A \rangle \in ALL_{NFA}$  iff  $f(\langle A \rangle) \in VC$ .

<sup>1</sup>As usual we ignore the issue of illegal encodings given as inputs.

**Computability:** The reduction is computable since  $ALL_{NFA} \in R$ , so we can determine whether the input belongs or not to  $ALL_{NFA}$ , and then we need to output one of two constant outputs which is also clearly computable.

## Solution 6:

The language  $L$  is empty and hence belongs to  $R$ . Indeed, if  $M$  is a TM that accepts some word of the form  $\sigma\sigma$  in a single step, then  $q_0 = q_{acc}$  or  $\delta(q_0, \sigma) = (q_{acc}, \gamma, d)$  for some  $\gamma \in \Gamma$  and  $d \in \{L, R\}$ . In any case,  $M$  accepts all words that begin with the letter  $\sigma$  in at most a single step. In particular, it could not be that in addition  $M$  does not halt on  $\sigma\sigma\sigma\sigma$ . Thus, for all  $\langle M \rangle$  and  $\sigma$ , either  $M$  does not accept  $\sigma\sigma$  in a single step, or  $M$  halts on  $\sigma\sigma\sigma\sigma$ . In either case  $\langle M \rangle \notin L$ , and hence  $L = \emptyset \in R$ .

## Solution 7:

The claim is correct - if  $SAT \leq_p POINT - SAT$  then  $P = NP$ . Let us first prove that  $POINT - SAT \in P$ . The key observation for this is that for a formula  $\varphi$  above  $n \geq 1$  variables, there are exactly  $n$  point assignments. Accordingly, we can go over all  $n$  assignments, and check if one of them satisfies  $\varphi$ .

**Construction:** We saw that there exists a polynomial time verifier  $V$ , that on input  $\langle \varphi, \vec{a} \rangle$  verifies that  $\vec{a} \in \{0, 1\}^n$  is a satisfying assignment for  $\varphi$ . Hence, all we need to do in order to solve  $POINT - SAT$  is to iterate over all  $i = 1, 2, \dots, n$ , and for each  $i$  call  $V(\varphi, \vec{a}^i)$ , where  $\vec{a}^i$  is the  $i$ 'th point assignment, and to accept if one of the calls was accepting.

**Complexity:** Each of the point assignments  $\vec{a}^i$  can be computed in polynomial time, each call for  $V$  takes polynomial time, and there are polynomially many calls, so the overall time complexity is polynomial.

**Correctness:**  $\langle \varphi \rangle \in POINT - SAT$  iff  $\varphi$  has a satisfying point assignment iff the machine  $M$  from the construction finds a point assignment that satisfies  $\varphi$  iff  $\langle \varphi \rangle \in L(M)$ .

**$SAT \leq_p POINT - SAT$  implies  $NP = P$ :** Recall now that  $SAT$  is  $NP$ -hard, and consider a language  $L \in NP$ . If  $SAT \leq_p POINT - SAT$ , then  $L \leq_p SAT \leq_p POINT - SAT$ , and by the transitivity of polynomial time reductions we have  $L \leq_p POINT - SAT$ . As  $POINT - SAT \in P$ , we conclude by the polynomial-time reduction theorem that  $L \in P$ . Hence, if  $SAT \leq_p POINT - SAT$  then  $NP \subseteq P$ . Since the other containment trivially holds, we have that  $SAT \leq_p POINT - SAT$  implies  $NP = P$ .

## Solution 8:

The language is  $NPC$ . Since  $NP$  is closed under intersection, and  $L = VC \cap IS$ , we clearly have that  $L \in NP$ . So it is only left to prove that  $L$  is  $NP$ -hard. We do so by showing that  $VC \leq_p L$  as follows.

**Construction:** Given input  $\langle G, k \rangle$  for VC where  $G = \langle V, E \rangle$  we distinguish between two cases. In the first case  $k > |V|$ , and then the answer for  $\langle G, k \rangle \in VC$  is clearly yes, and we output  $\langle \hat{G}, 1 \rangle$  where  $\hat{G} = \langle \{1\}, \emptyset \rangle$  is a graph with a single vertex and no edges. Otherwise,  $k \leq |V|$ , and then we output  $\langle G', k \rangle$  where  $G' = \langle V', E \rangle$  is the graph obtained from  $G$  by adding  $k$  new isolated vertices. Thus,  $V' = V \cup V''$  where  $V'' \cap V = \emptyset$  and  $|V''| = k$ .

**Complexity:** Checking whether  $k \leq |V|$  takes polynomial time in  $\langle k \rangle$  and  $|V|$ , and hence polynomial in  $\langle G, k \rangle$ . Then, in the first case, i.e. when  $k > |V|$ , it is clear that writing the constant description of  $\hat{G}$  takes constant time.

Otherwise, when  $k \leq |V|$ , we add  $k$  new vertices to a graph, and this can be done in polynomial time in  $|\langle G, k \rangle|$ . Indeed, we need to increment the maximal vertex encoding  $k = O(|V|)$  times, and add each one of the  $k$  new encodings to the vertex list. The maximal vertex of  $G'$  is described with  $\log(|V| + k) = O(|V|)$  bits, and so constructing  $k$  new vertices take at most  $O(k \log(|V|)^2) = O(|V| \log(|V|)^2)$  which is polynomial in  $|\langle G, k \rangle|$ .

**Correctness:** The case that  $k \geq |V|$  is straight forward as indeed  $\langle G, k \rangle \in VC$  and  $\langle \hat{G}, 1 \rangle \in L$ .

Otherwise, when  $k \leq |V|$ , note that if  $C \subseteq V'$  is a vertex cover of  $G'$ , then so is  $C \cap V$  a vertex cover. Indeed, vertices in  $V''$  are isolated and so do not contribute to the covering of  $E$ . So  $G'$  has a vertex cover of size  $k$  iff it has a vertex cover  $C \subseteq V$  of size  $k$ , iff  $G$  has a vertex cover of size  $k$ . Finally, it is clear that  $G'$  has an independent set of size  $k$ , namely  $V''$ , and hence  $G$  has a vertex cover of size  $k$  iff  $G'$  has a vertex cover and an independent set both of size  $k$ . That is  $\langle G, k \rangle \in VC$  iff  $\langle G', k \rangle \in L$  as required.

## Solution 9:

The claim is correct, the class NPC is closed under the *AtLeastTwo* operator.

**First direct. (NP is closed under AtLeastTwo):** Let  $\Sigma$  be an alphabet that does not include  $\#$ , and consider a language  $L \subseteq \Sigma^*$  that is in NP. It is not hard to check that the following identity holds:

$$AtLeastTwo(L) = (\Sigma^* \cdot \#)^* \cdot L \cdot \{\#\} \cdot (\Sigma^* \cdot \#)^* \cdot L \cdot (\# \cdot \Sigma^*)^*$$

Observe that  $(\Sigma \cdot \#)^*$ ,  $(\# \cdot \Sigma^*)^*$  and  $\{\#\}$  are all regular and hence belong to  $DTIME(n) \subseteq P \subseteq NP$ , and by assumption  $L \in NP$ . Thus,  $AtLeastTwo(L)$  is a finite concatenation of NP languages, and by the closure of NP to finite concatenations we conclude that  $AtLeastTwo(L) \in NP$ .

**Second direction. (If  $L$  is NP – hard then so is  $AtLeastTwo(L)$ ):** If  $L$  is NP – hard then  $w \mapsto w\#w$  is a reduction from  $L$  to  $AtLeastTwo(L)$  and is clearly computable in polynomial time. Thus, if  $L$  is NP – hard then so is  $AtLeastTwo(L)$ .